Experiment No: 07

Python for Data 24: Hypothesis Testing

back to index

Point estimates and confidence intervals are basic inference tools that act as the foundation for another inference technique: statistical hypothesis testing. Statistical hypothesis testing is a framework for determining whether observed data deviates from what is expected. Python's scipy.stats library contains an array of functions that make it easy to carry out hypothesis tests.

Hypothesis Testing Basics

Statistical hypothesis tests are based a statement called the null hypothesis that assumes nothing interesting is going on between whatever variables you are testing. The exact form of the null hypothesis varies from one type test to another: if you are testing whether groups differ, the null hypothesis states that the groups are the same. For instance, if you wanted to test whether the average age of voters in your home state differs from the national average, the null hypothesis would be that there is no difference between the average ages.

The purpose of a hypothesis test is to determine whether the null hypothesis is likely to be true given sample data. If there is little evidence against the null hypothesis given the data, you accept the null hypothesis. If the null hypothesis is unlikely given the data, you might reject the null in favor of the alternative hypothesis: that something interesting is going on. The exact form of the alternative hypothesis will depend on the specific test you are carrying out. Continuing with the example above, the alternative hypothesis would be that the average age of voters in your state does in fact differ from the national average.

Once you have the null and alternative hypothesis in hand, you choose a significance level (often denoted by the Greek letter α.). The significance level is a probability threshold that determines when you reject the null hypothesis. After carrying out a test, if the probability of getting a result as extreme as the one you observe due to chance is lower than the significance level, you reject the null hypothesis in favor of the alternative. This probability of seeing a result as extreme or more extreme than the one observed is known as the p-value.

The T-test is a statistical test used to determine whether a numeric data sample of differs significantly from the population or whether two samples differ from one another.

One-Sample T-Test

A one-sample t-test checks whether a sample mean differs from the population mean. Let's create some dummy age data for the population of voters in the entire country and a sample of voters in Minnesota and test the whether the average age of voters Minnesota differs from the population:

```
%matplotlib inline

import numpy as np
import pandas as pd
import scipy.stats as stats
import matplotlib.pyplot as plt
import math
```

```
np.random.seed(6)

population_ages1 = stats.poisson.rvs(loc=18, mu=35, size=150000)
population_ages2 = stats.poisson.rvs(loc=18, mu=10, size=100000)
population_ages = np.concatenate((population_ages1, population_ages2))

minnesota_ages1 = stats.poisson.rvs(loc=18, mu=30, size=30)
minnesota_ages2 = stats.poisson.rvs(loc=18, mu=10, size=20)
minnesota_ages = np.concatenate((minnesota_ages1, minnesota_ages2))

print( population_ages.mean() )
print( minnesota_ages.mean() )
```

```
43.000112
39.26
```

Notice that we used a slightly different combination of distributions to generate the sample data for Minnesota, so we know that the two means are different. Let's conduct a t-test at a 95% confidence level and see if it correctly rejects the null hypothesis that the sample comes from the same distribution as the population. To conduct a one sample t-test, we can the stats.ttest_1samp() function:

```
stats.ttest_1samp(a = minnesota_ages,               # Sample data
                  popmean = population_ages.mean())  # Pop mean
```

```
TtestResult(statistic=np.float64(-2.5742714883655027),
pvalue=np.float64(0.013118685425061678), df=np.int64(49))
```

The test result shows the test statistic "t" is equal to -2.574. This test statistic tells us how much the sample mean deviates from the null hypothesis. If the t-statistic lies outside the quantiles of the t-distribution corresponding to our confidence level and degrees of freedom, we reject the null hypothesis. We can check the quantiles with stats.t.ppf():

```
stats.t.ppf(q=0.025,  # Quantile to check
            df=49)  # Degrees of freedom
```

```
np.float64(-2.0095752371292397)
```

```
stats.t.ppf(q=0.975,  # Quantile to check
            df=49)  # Degrees of freedom
```

```
np.float64(2.0095752371292397)
```

We can calculate the chances of seeing a result as extreme as the one we observed (known as the p-value) by passing the t-statistic in as the quantile to the stats.t.cdf() function:

```
stats.t.cdf(x= -2.5742,      # T-test statistic
               df= 49) * 2   # Multiply by two for two tailed test *
```

```
np.float64(0.013121066545690117)
```

Note: The alternative hypothesis we are checking is whether the sample mean differs (is not equal to) the population mean. Since the sample could differ in either the positive or negative direction we multiply the by two.

Notice this value is the same as the p-value listed in the original t-test output. A p-value of 0.01311 means we'd expect to see data as extreme as our sample due to chance about 1.3% of the time if the null hypothesis was true. In this case, the p-value is lower than our significance level α (equal to 1-conf.level or 0.05) so we should reject the null hypothesis. If we were to construct a 95% confidence interval for the sample it would not capture population mean of 43:

```
sigma = minnesota_ages.std()/math.sqrt(50)  # Sample stdev/sample size

stats.t.interval(0.95,                        # Confidence level
                 df = 49,                     # Degrees of freedom
                 loc = minnesota_ages.mean(), # Sample mean
                 scale= sigma)                # Standard dev estimate
```

```
(np.float64(36.36966907692507), np.float64(42.150330923074925))
```

On the other hand, since there is a 1.3% chance of seeing a result this extreme due to chance, it is not significant at the 99% confidence level. This means if we were to construct a 99% confidence interval, it would capture the population mean:

```
stats.t.interval(alpha = 0.99,                # Confidence level
                 df = 49,                     # Degrees of freedom
                 loc = minnesota_ages.mean(), # Sample mean
                 scale= sigma)                # Standard dev estimate
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_1924/718392128.py in <cell line: 0>()
----> 1 stats.t.interval(alpha = 0.99,                # Confidence level
      2                  df = 49,                     # Degrees of freedom
      3                  loc = minnesota_ages.mean(), # Sample mean
      4                  scale= sigma)                # Standard dev
estimate

TypeError: rv_generic.interval() missing 1 required positional argument:
'confidence'
```

With a higher confidence level, we construct a wider confidence interval and increase the chances that it captures to true mean, thus making it less likely that we'll reject the null hypothesis. In this case, the p-value of 0.013 is greater than our significance level of 0.01 and we fail to reject the null hypothesis.

Two-Sample T-Test

A two-sample t-test investigates whether the means of two independent data samples differ from one another. In a two-sample test, the null hypothesis is that the means of both groups are the same. Unlike the one sample-test where we test against a known population parameter, the two sample test only involves sample means. You can conduct a two-sample t-test by passing with the stats.ttest_ind() function. Let's generate a sample of voter age data for Wisconsin and test it against the sample we made earlier:

```
np.random.seed(12)
wisconsin_ages1 = stats.poisson.rvs(loc=18, mu=33, size=30)
wisconsin_ages2 = stats.poisson.rvs(loc=18, mu=13, size=20)
wisconsin_ages = np.concatenate((wisconsin_ages1, wisconsin_ages2))

print( wisconsin_ages.mean() )
```

```
42.8
```

```
stats.ttest_ind(a= minnesota_ages,
                b= wisconsin_ages,
                equal_var=False)    # Assume samples have equal variance?
```

```
TtestResult(statistic=np.float64(-1.7083870793286842),
pvalue=np.float64(0.09073104343957748), df=np.float64(97.9724575497005))
```

The test yields a p-value of 0.0907, which means there is a 9% chance we'd see sample data this far apart if the two groups tested are actually identical. If we were using a 95% confidence level we would fail to reject the null hypothesis, since the p-value is greater than the corresponding significance level of 5%.

Paired T-Test

The basic two sample t-test is designed for testing differences between independent groups. In some cases, you might be interested in testing differences between samples of the same group at different points in time. For instance, a hospital might want to test whether a weight-loss drug works by checking the weights of the same group patients before and after treatment. A paired t-test lets you check whether the means of samples from the same group differ.

We can conduct a paired t-test using the scipy function stats.ttest_rel(). Let's generate some dummy patient weight data and do a paired t-test:

```
np.random.seed(11)

before= stats.norm.rvs(scale=30, loc=250, size=100)

after = before + stats.norm.rvs(scale=5, loc=-1.25, size=100)

weight_df = pd.DataFrame({"weight_before":before,
                          "weight_after":after,
                          "weight_change":after-before})

weight_df.describe()             # Check a summary of the data
```

	weight_before	weight_after	weight_change
count	100.000000	100.000000	100.000000
mean	250.345546	249.115171	-1.230375
std	28.132539	28.422183	4.783696
min	170.400443	165.913930	-11.495286
25%	230.421042	229.148236	-4.046211
50%	250.830805	251.134089	-1.413463
75%	270.637145	268.927258	1.738673
max	314.700233	316.720357	9.759282

The summary shows that patients lost about 1.23 pounds on average after treatment. Let's conduct a paired t-test to see whether this difference is significant at a 95% confidence level:

```
stats.ttest_rel(a = before,
                b = after)
```

```
TtestResult(statistic=np.float64(2.5720175998568284),
pvalue=np.float64(0.011596444318439857), df=np.int64(99))
```

Type I and Type II Error

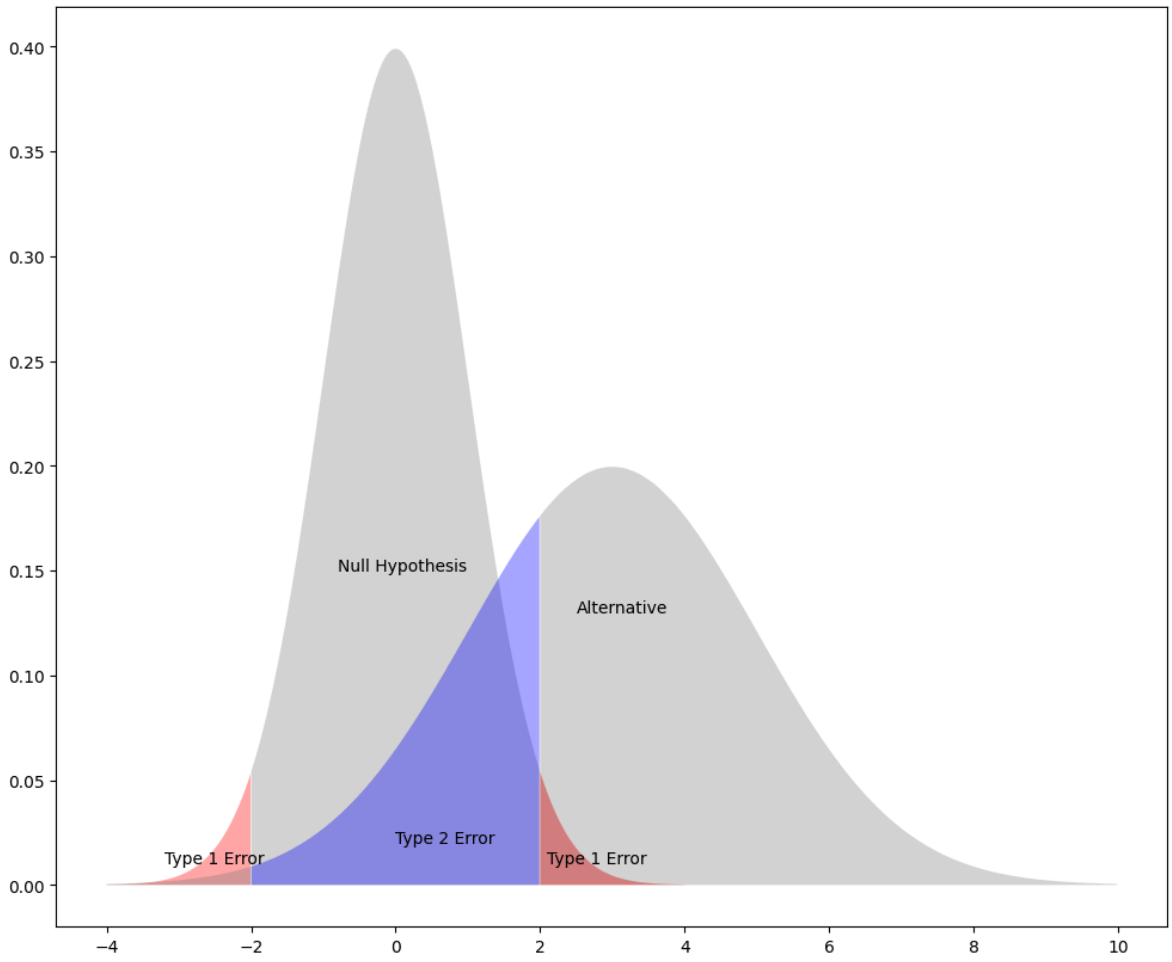
The result of a statistical hypothesis test and the corresponding decision of whether to reject or accept the null hypothesis is not infallible. A test provides evidence for or against the null hypothesis and then you decide whether to accept or reject it based on that evidence, but the evidence may lack the strength to arrive at the correct conclusion. Incorrect conclusions made from hypothesis tests fall in one of two categories: type I error and type II error.

Type I error describes a situation where you reject the null hypothesis when it is actually true. This type of error is also known as a "false positive" or "false hit". The type 1 error rate is equal to the significance level α, so setting a higher confidence level (and therefore lower alpha) reduces the chances of getting a false positive.

Type II error describes a situation where you fail to reject the null hypothesis when it is actually false. Type II error is also known as a "false negative" or "miss". The higher your confidence level, the more likely you are to make a type II error.

Let's investigate these errors with a plot:

```
plt.figure(figsize=(12,10))


plt.fill_between(x=np.arange(-4,-2,0.01),
                 y1= stats.norm.pdf(np.arange(-4,-2,0.01)) ,
                 facecolor='red',
                 alpha=0.35)

plt.fill_between(x=np.arange(-2,2,0.01),
                 y1= stats.norm.pdf(np.arange(-2,2,0.01)) ,
                 facecolor='grey',
                 alpha=0.35)

plt.fill_between(x=np.arange(2,4,0.01),
                 y1= stats.norm.pdf(np.arange(2,4,0.01)) ,
                 facecolor='red',
                 alpha=0.5)

plt.fill_between(x=np.arange(-4,-2,0.01),
                 y1= stats.norm.pdf(np.arange(-4,-2,0.01),loc=3, scale=2) ,
                 facecolor='grey',
                 alpha=0.35)

plt.fill_between(x=np.arange(-2,2,0.01),
                 y1= stats.norm.pdf(np.arange(-2,2,0.01),loc=3, scale=2) ,
                 facecolor='blue',
                 alpha=0.35)

plt.fill_between(x=np.arange(2,10,0.01),
                 y1= stats.norm.pdf(np.arange(2,10,0.01),loc=3, scale=2),
                 facecolor='grey',
                 alpha=0.35)

plt.text(x=-0.8, y=0.15, s= "Null Hypothesis")
plt.text(x=2.5, y=0.13, s= "Alternative")
plt.text(x=2.1, y=0.01, s= "Type 1 Error")
plt.text(x=-3.2, y=0.01, s= "Type 1 Error")
plt.text(x=0, y=0.02, s= "Type 2 Error");
```



In the plot above, the red areas indicate type I errors assuming the alternative hypothesis is not different from the null for a two-sided test with a 95% confidence level.

The blue area represents type II errors that occur when the alternative hypothesis is different from the null, as shown by the distribution on the right. Note that the Type II error rate is the area under the alternative distribution within the quantiles

determined by the null distribution and the confidence level. We can calculate the type II error rate for the distributions above as follows:

```
lower_quantile = stats.norm.ppf(0.025)  # Lower cutoff value
upper_quantile = stats.norm.ppf(0.975)  # Upper cutoff value

# Area under alternative, to the left the lower cutoff value
low = stats.norm.cdf(lower_quantile,
                     loc=3,
                     scale=2)

# Area under alternative, to the left the upper cutoff value
high = stats.norm.cdf(upper_quantile,
                      loc=3,
                      scale=2)

# Area under the alternative, between the cutoffs (Type II error)
high-low
```

```
np.float64(0.294956061112323)
```

With the normal distributions above, we'd fail to reject the null hypothesis about 30% of the time because the distributions are close enough together that they have significant overlap.

Statistical Power

The power of a statistical test is the probability that the test rejects the null hypothesis when the alternative is actually different from the null. In other words, power is the probability that the test detects that there is something interesting going on when there actually *is* something interesting going on. Power is equal to one minus the type II error rate. The power of a statistical test is influenced by:

1. The significance level chosen for the test.
2. The sample size.
3. The effect size of the test.

When choosing a significance level for a test, there is a trade-off between type I and type II error. A low significance level, such as 0.01 makes a test unlikely to have type I errors (false positives), but more likely to have type II errors (false negatives) than a test with larger value of the significance level α. A common convention is that a statistical tests should have a power of at least 0.8.

A larger sample size reduces the uncertainty of the point estimate, causing the sample distribution to narrow, resulting in lower type II error rates and increased power.

Effect size is a general term that describes a numeric measure of the size of some phenomenon. There are many different effect size measurements that arise in different contexts. In the context of the T-test, a simple effect size is the difference between the means of the samples. This number can be standardized by dividing by the standard deviation of the population or the pooled standard deviation of the samples. This puts the size of the effect in terms of standard deviations, so a standardized effect size of 0.5 would be interpreted as one sample mean being 0.5 standard deviations from another (in general 0.5 is considered a "large" effect size).

Since statistical power, the significance level, the effect size and the sample size are related, it is possible to calculate any one of them for given values of the other three. This can be an important part of the process of designing a hypothesis test and analyzing results. For instance, if you want to conduct a test with a given significance level (say the standard 0.05) and power (say the standard 0.8) and you are interested in a given effect size (say 0.5 for standardized difference between sample means), you could use that information to determine how large of a sample size you need.

In python, the statsmodels library contains functions to solve for any one parameter of the power of T-tests. Use statsmodels.stats.power.tt_solve_power for one sample t-tests and statsmodels.stats.power.tt_ind_solve_power for a two sample t-test. Let's check the sample size we should use need to use given the standard parameter values above for a one sample t-test:

```
from statsmodels.stats.power import tt_solve_power

tt_solve_power(effect_size = 0.5,
               alpha = 0.05,
               power = 0.8)
```

```
33.36713118431778
```

In this case, we would want a sample size of at least 34 to make a study with the desired power and signifiance level capable of detecting a large effect size.

1.What is need for Hypothesis Testing ?

Hypothesis testing is a framework used to determine whether observed data deviates from what is expected. It helps assess whether a statement about a population (the null hypothesis) is likely to be true given sample data. If there's sufficient evidence against the null hypothesis, it is rejected in favor of an alternative hypothesis, suggesting that something 'interesting' is happening.

2.With a neat diagram show important steps in hypothesis Testing.

flowchart TD A[Start] --> B{Formulate Hypotheses
H0: Null, Ha: Alternative} B --> C{Choose Significance Level (α)} C --> D[Collect Sample Data] D --> E{Calculate Test Statistic} E --> F{Determine p-value} F --> G{Is p-value < α?} G -- Yes --> H[Reject Null Hypothesis (H0)] G -- No --> I[Fail to Reject Null Hypothesis (H0)] H --> J[Draw Conclusion] I --> J J --> K[End] Explanation of Steps:

Formulate Hypotheses: Define the null hypothesis (H0), which states there is no effect or no difference, and the alternative hypothesis (Ha), which states there is an effect or difference. Choose Significance Level (α): Set a threshold for deciding whether to reject the null hypothesis, commonly 0.05 (5%). This represents the probability of a Type I error. Collect Sample Data: Gather relevant data through experiments or observations. Calculate Test Statistic: Compute a value from the sample data (e.g., t-statistic, z-statistic) that measures how far the sample results deviate from what is expected under the null hypothesis. Determine p-value: Calculate the probability of observing a test statistic as extreme as, or more extreme than, the one calculated from your sample, assuming the null hypothesis is true. Compare p-value to α: Compare the calculated p-value with the chosen significance level (α). Make Decision: If p-value < α, reject the null hypothesis. There is statistically significant evidence to support the alternative hypothesis. If p-value ≥ α, fail to reject the null hypothesis. There is not enough evidence to conclude that the alternative hypothesis is true.

3.What is a student t distribution?

The Student's t-distribution, often referred to simply as the t-distribution, is a probability distribution that arises when estimating the mean of a normally distributed population in situations where the sample size is small and the population standard deviation is unknown. It is similar in shape to the normal distribution but has fatter tails, meaning it has a higher probability for extreme values. As the sample size increases, the t-distribution approaches the normal distribution. It is widely used in hypothesis testing, particularly in t-tests, to make inferences about population means from sample data.

4.What is level of significance ?

What role t plays in hypothesis Testing The level of significance (α) is a probability threshold that determines when you reject the null hypothesis. It represents the maximum probability of making a Type I error (incorrectly rejecting a true null hypothesis). Common significance levels are 0.05 (5%) or 0.01 (1%). If the p-value from your test is less than α, you reject the null hypothesis.

The t-statistic (or t-value) plays a crucial role in hypothesis testing, especially in t-tests. It measures how many standard errors the sample mean is away from the population mean (or the hypothesized population mean in a one-sample t-test, or the

difference between two sample means in a two-sample t-test). In essence, it quantifies the difference between your observed sample data and what you would expect to see if the null hypothesis were true, relative to the variability in your data. A larger absolute value of the t-statistic indicates a greater difference, making it more likely to reject the null hypothesis.

Hypothesis Test Design: Average Age of TE IT Students To test if the average age of students in a TE IT class is 19, we'll follow these steps:

1. Formulate Hypotheses Null Hypothesis (H0): The average age of students in the TE IT class is 19 years ($\mu=19$). Alternative Hypothesis (Ha): The average age of students in the TE IT class is not 19 years ($\mu\neq19$). This is a two-tailed test because we are interested in whether the age is different from 19, not specifically greater or less than 19.
2. Choose Significance Level (α) We need to decide on a significance level, which is the probability of rejecting the null hypothesis when it is actually true (Type I error). A common choice is $\alpha = 0.05$.

$\alpha = 0.05$ (or 5%) This means we are willing to accept a 5% chance of incorrectly concluding that the average age is not 19, when in reality it is. 3. Collect Sample Data To conduct the test, we would need to collect a random sample of ages from the students in the TE IT class. For this example, let's create some dummy data.

```
import numpy as np
import scipy.stats as stats

# Set a seed for reproducibility
np.random.seed(42)

# Generate dummy data for student ages in a TE IT class
# Let's assume a true mean of around 19, but with some variation
sample_size = 30
actual_mean = 19.5 # Let's assume the true mean is slightly different for de
std_dev = 2
student_ages = np.random.normal(loc=actual_mean, scale=std_dev, size=sample_

print(f"Sample Mean Age: {student_ages.mean():.2f}")
print(f"Sample Standard Deviation: {student_ages.std():.2f}")
```

```
Sample Mean Age: 19.12
Sample Standard Deviation: 1.77
```

4. Choose the Appropriate Statistical Test Since we are comparing a sample mean to a known population mean (or hypothesized mean, 19 in this case) and the population standard deviation is unknown (which is typical for real-world scenarios), a One-Sample T-Test is appropriate.

5. Calculate Test Statistic and p-value We will use scipy.stats.ttest_1samp to perform the one-sample t-test. This function returns the t-statistic and the two-tailed p-value directly.

```
hypothesized_mean = 19

t_statistic, p_value = stats.ttest_1samp(a=student_ages, popmean=hypothesize

print(f"T-statistic: {t_statistic:.3f}")
print(f"P-value: {p_value:.4f}")
```

```
T-statistic: 0.376
P-value: 0.7093
```

6. Make a Decision and Draw a Conclusion We compare the calculated p-value to our chosen significance level (α = 0.05).

If p-value < α: Reject the null hypothesis. If p-value ≥ α: Fail to reject the null hypothesis. Based on the output from the previous cell, we can make our decision.

6.Check if you can accept Ho at alpha 95% ?

Match your conclusions with python functions. To check if we can accept the null hypothesis (H0) at a 95% confidence level (meaning alpha = 0.05), we need to compare the calculated p-value from our t-test with this alpha value. If the p-value is less than alpha, we reject H0; otherwise, we fail to reject H0.

From the previous ttest_1samp function, we obtained a p-value of 0.7093.